

Reth GPU performance — 1 / 2 / 4 / 8 RTX 5090 (JIT-by-default)

Benchmark of the reth ELF (zkm-gpu-perf --program reth --stage wrap) on the ant-5090-2 box (8× NVIDIA RTX 5090, 124 logical cores, 925 GB RAM, 36 GB visible per GPU). Runs use the JIT-by-default executor (feat/upgrade-plonky3 Ziren) end-to-end through prove + verify, with the GPU prover from feat/gpu-basefold-dispatch ziren-gpu. Trace generation uses TRACE_GEN_WORKERS=16, recursion_opts.shard_batch_size = 4, GPUs 2-7 selected via CUDA_VISIBLE_DEVICES.

Workload

Field	Value
Program	reth (perf/mipsel-zkm-zkvm-elf/reth in ziren-gpu)
Cycles	419,960,677
Shards	189
ELF size	5.2 MB
Stdin	9.1 MB (single block + state proofs)
Proof size	338,288,443 bytes (uncompressed Groth16-ready bn254 wrap)

Results

RECURSION_SHARD_BATCH_SIZE (SBS) controls the number of concurrent prover-submit threads in compress_multi_gpu. The default now scales with GPU count: (gpu_count * 2).clamp(4, 8) — see crates/stark/src/opts.rs::ZKMProverOpts::gpu. Override via RECURSION_SHARD_BATCH_SIZE.

GPUs	SBS	Core (s)	Compress (s)	Shrink (s)	Wrap (s)	Total (s)	Core kHz	Δ vs SBS=4
1	4	106.5	52.9	0.43	1.11	160.9	3,944	baseline
2	4	64.7	46.0	0.45	1.21	112.4	6,488	baseline
4	4	58.3	42.1	0.46	1.02	101.9	7,210	baseline
4	8	63.1	32.5	0.41	1.08	97.1	6,654	−4.8 s
8	4	56.4	42.0	0.44	1.08	99.9	7,452	baseline
8	8	59.8	33.1	0.42	1.12	94.4	7,027	−5.5 s
8	12	62.8	33.6	0.42	1.14	98.0	6,687	−1.9 s
8	8 + RMTPD=2	61.3	36.4	0.45	1.09	99.2	6,851	−0.7 s

Speedup (with the new SBS default)

- 1 → 2 GPU: 1.43× total (1.65× core)
- 1 → 4 GPU: 1.66× total (1.83× core)
- 2 → 4 GPU: 1.16× total (compress 46 → 32 s with SBS=8)
- 4 → 8 GPU: 1.03× total — was 1.02× at SBS=4. Core plateaus at 4 workers (CPU trace-gen floor); compress now scales 1:1 with the GPU pool instead of saturating at the SBS=4 ceiling.

Why SBS matters

`compress_multi_gpu` spawns `recursion_opts.shard_batch_size` worker threads. Each thread pulls one (record, traces) tuple off `record_and_trace_rx`, calls `pool.submit()` → `setup()` + `commit()` + `open()` on a GPU pool worker, blocks on the result, then loops. With `SBS=4` and 8 GPUs only 4 of 8 GPU pool workers ever have work in flight.

- **8 GPU SBS=4 → SBS=8** unlocks the idle 4 GPUs. Compress drops 42.0 s → 33.1 s (-21 %) and total 99.9 → 94.4 (-5.5 %).
- **4 GPU SBS=4 → SBS=8** oversubscribes 2× per GPU. Per-shard CPU prep (recursion-program build, `setup()` on host side, `generate_dependencies`) overlaps with another shard's GPU work. Compress drops 42.1 → 32.5 s (-23 %) and total 101.9 → 97.1 (-4.7 %).
- **8 GPU SBS=12** plateaus then regresses (worker threads contend on state mutexes; core 56.4 → 62.8 s).
- **ZKM_GPU_RECURSION_MAX_TASKS_PER_DEVICE=2** also regresses (GPU memory pressure from two concurrent in-flight tasks per device).
- **1 GPU SBS=8** OOMs on real reth shards (8 `setup()` / `commit()` / `open()` concurrent allocations on a 32 GB card). The new default `((gpu_count * 2).clamp(4, 8))` keeps 1-2 GPU at `SBS=4` for safety and bumps 4-8 GPU to `SBS=8` for the win.

JIT-by-default vs interpreter (4 GPU, identical proof output)

Both modes produce `cycles=419,960,677`, `proofBytes=338,288,443`.

Mode	Core (s)	Compress (s)	Shrink (s)	Wrap (s)	Total (s)	Core kHz
JIT-by-default	61.5	41.9	0.46	1.04	105.0	6,823
Interpreter	58.7	42.3	0.44	1.11	102.5	7,160

The two are within run-to-run variance. The JIT wins big on **execute-only** workloads (tendermint 1.8 s → 0.27 s, 6.7×; Reth single-block 25 s → 5 s). Once the GPU prover is engaged, FRI commit / multi-shard recursion / trace LDE dominate, so the executor speedup is invisible at the e2e level. Both modes still satisfy every per-stage verifier — the JIT-produced trace is mathematically equivalent.

End-to-end correctness

Every configuration successfully runs **prove core → compress → shrink → wrap_bn254 → verify core → verify compress → verify shrink → verify wrap** with wrapping successful. The JIT-produced execution trace is mathematically validated through every recursion layer up to BN254. The wrap-stage verifier (`wrap_bn254::verify`) takes ~70 ms.

JIT executor changes (this run)

The executor now runs JIT-by-default for every workload (interpreter is the fallback for unsupported opcodes only). Key fixes vs the prior doc:

- **Memfd-COW for unconstrained blocks** — host buffer switched from anonymous mmap to `memfd_create` + `MAP_SHARED`; `ENTER_UNCONSTRAINED` mmap's a private COW view of the same fd, `EXIT` munmaps it. Mirrors SP1's `crates/core/jit/src/context.rs::enter_unconstrained`. Without this, Reth panicked at ~108 M cycles with `NodeNotResolved`.

- **Per-precompile sync table** — every curve op (SECP256K1 / SECP256R1 / BN254 / BLS12381 / ED25519), Fp/Fp2 op, KECCAK_SPONGE, U256_MUL, U256xU2048_MUL, POSEIDON2_PERMUTE has explicit input/output byte ranges synced through the bridge. Caught a 60 % Reth-block failure rate (U256_MUL didn't sync the modulus at A1+32; KECCAK_SPONGE used a wrong fixed length instead of reading it from mem[A1+64]).
- **clk_bump = 1** — JIT's state.global_clk now matches the interpreter's exactly (previously bumped by 4 per instruction).
- **No-pool mem_fd** — Drop always closes the fd to avoid stale page-cache contents leaking into the next run's COW snapshot.

The executor's JIT vs interp output is now **byte-identical** (cycles + public-values + first-16 bytes) across all 12 standard shape-bin workloads.

Bottleneck analysis

1. Compress stage CPU work (~42 s at 4 GPU)

Compress is now ~41 % of wall time at 4 GPUs and is the next obvious target. The remaining cost is mostly CPU-side:

- Per-shard recursion-program build + setup() (CPU).
- Dummy witness construction in dummy_vk_and_shard_proof (CPU).
- Jagged-fingerprint computation per shard (CPU).
- make_merkle_proofs lookup + Merkle path open (CPU).

The shard-level basefold prover code path (formerly the WHIR fast path) is currently dead — use_whir = false because BasefoldShardVerifier still fails round-0 sumcheck on real shards. Re-enabling it as a deliberate selector once that's fixed would let compress amortise more work to the GPU.

2. Core stage scales 1 → 2 GPU and plateaus

Core dropped 106.5 s → 64.7 s with 2 GPUs (1.65×) but only 64.7 s → 58.3 s with 4 GPUs (1.11×). Per-shard core cost falls from ~563 ms (1 GPU) to ~342 ms (2 GPU) and stays there.

Plateau hypotheses (in priority order):

1. **CPU-side trace generation saturates.** TRACE_GEN_WORKERS=16 is already 16× the legacy default; bumping further may help.
2. **PCIe bandwidth between host and GPUs.** Reth shards are large (multi-MB traces) and core dispatch is one-trace-at-a-time per worker.
3. **CUDA stream serialisation in the per-GPU prover queue.** Each GPU gets its own MultiGpuDevicePool worker but inside that worker FRI commit / sumcheck phases serialise.

3. Shrink + Wrap are fixed at ~1.5 s

Single-GPU operations by design. No parallelism opportunity.

Where to push next

The SBS default change above lands the cheapest win. Remaining levers:

Lever	Estimated wall reduction	Effort
Cache per-shard pk across shape-equivalent shards (skip setup() GPU upload on cache hit)	~5-10 s (compress)	medium
Fix BasefoldShardVerifier round-0 + re-enable shard-level basefold	~15 s	high
Increase TRACE_GEN_WORKERS to 24-32 with a contention audit on the per-shard state mutex	~2-5 s (core)	low — needs profiling first
Pin shard checkpoints in memory (skip reload per shard)	~5 s	medium
Wider PCIe (Gen 5 vs current)	hardware-bound	n/a

TRACE_GEN_WORKERS=32 was tested and gave a small core regression (58.3 → 62.5 s at 4 GPU) — the bottleneck is the per-shard state.lock() mutex inside the trace-gen worker loop, not the worker count. Profile that path before adding more workers.

Reproducing

```
# On ant-5090-2 (Ziren on feat/upgrade-plonky3 with JIT-by-default,
# ziren-gpu on feat/gpu-basefold-dispatch).
cd /home/ubuntu/sd/ziren-gpu
export PATH=/usr/local/cuda/bin:/usr/local/go/bin:$PATH
source ~/.zkm-toolchain/env

# Use a known-free GPU set and pin all the prover knobs.
export ZIREN_USE_BASEFOLD=1 ZIREN_BASEFOLD_GPU=1 VERIFY_VK=false
export ZKM_GPU_CORE_MAX_TASKS_PER_DEVICE=1
export ZKM_GPU_RECURSION_MAX_TASKS_PER_DEVICE=1
export DEFAULT_CHECKPOINTS_CHANNEL_CAPACITY=512
export SHAPE_CHECK_FREQUENCY=1024
export SHARD_SIZE=4194305
export TRACE_GEN_WORKERS=16
export RECORDS_AND_TRACES_CHANNEL_CAPACITY=16
mkdir -p /dev/shm/zkm-gpu-perf && export TMPDIR=/dev/shm/zkm-gpu-perf

# 1 GPU
CUDA_VISIBLE_DEVICES=2 ZKM_GPU_DEVICES=0 \
./target/release/zkm-gpu-perf --program reth --stage wrap

# 2 GPUs
CUDA_VISIBLE_DEVICES=2,3 ZKM_GPU_DEVICES=0,1 \
./target/release/zkm-gpu-perf --program reth --stage wrap

# 4 GPUs
CUDA_VISIBLE_DEVICES=2,3,4,5 ZKM_GPU_DEVICES=0,1,2,3 \
./target/release/zkm-gpu-perf --program reth --stage wrap

# 8 GPUs
CUDA_VISIBLE_DEVICES=0,1,2,3,4,5,6,7 ZKM_GPU_DEVICES=0,1,2,3,4,5,6,7 \
./target/release/zkm-gpu-perf --program reth --stage wrap
```

Drop `--skip-verify` (it's the default ON) to also run every per-stage verifier; total run time grows by $\sim 5\%$ for the verify pass.

Captured 2026-04-30

- Branch: `feat/gpu-basefold-dispatch (ziren-gpu)` $\leftrightarrow$ `feat/upgrade-plonky3 (Ziren)`
- Executor: JIT-by-default (`crates/core/jit`, `clk_bump=1`, `memfd-COW` unconstrained-block rollback).
- `vk_map.bin`: padded to $2^8 = 256$ entries (87 real vks + sentinels).
- Default proof system: FRI everywhere (host CPU + GPU paths share one `vk_map`). Re-introduce the BaseFold-shard prover as a deliberate selector — not env-gated — once round-0 sumcheck is resolved.